

HDOC Day-8 Assignment

Question 1: Number of Cones Made from a Cylinder

1. Problem Statement Identification

- **Problem Statement:** Determine the integer number of solid cones that can be created from a solid cylinder by computing exact volume division.
- **Input:** Cylinder Radius (r_1), Cylinder Height (h_1), Cone Radius (r_2), Cone Height (h_2).
- **Output:** Number of Cones (N).
- **Formulas:**

$$\text{Volume}_{\text{cylinder}} = \pi \cdot r_1^2 \cdot h_1$$

$$\text{Volume}_{\text{cone}} = \frac{1}{3} \cdot \pi \cdot r_2^2 \cdot h_2$$

$$N = \left\lfloor \frac{\text{Volume}_{\text{cylinder}}}{\text{Volume}_{\text{cone}}} \right\rfloor$$

2. Standard Test Case Table

Test Case	Input (r_cyl, h_cyl, r_cone, h_cone)	Expected Output	Actual Output	Result (Pass/Fail)
1	6 10 3 5	Number of Cones: 24	Number of Cones: 24	Pass
2	10 20 5 10	Number of Cones: 24	Number of Cones: 24	Pass
3	4 12 2 6	Number of Cones: 24	Number of Cones: 24	Pass
4	5 10 5 10	Number of Cones: 3	Number of Cones: 3	Pass

3. Algorithm

1. Start

2. Read r_1, h_1, r_2, h_2 .

3. If $r_1 \leq 0$ or $h_1 \leq 0$ or $r_2 \leq 0$ or $h_2 \leq 0$, print "Invalid Input" and stop.

4. Calculate $V_{\text{cylinder}} = \pi \cdot r_1^2 \cdot h_1$.

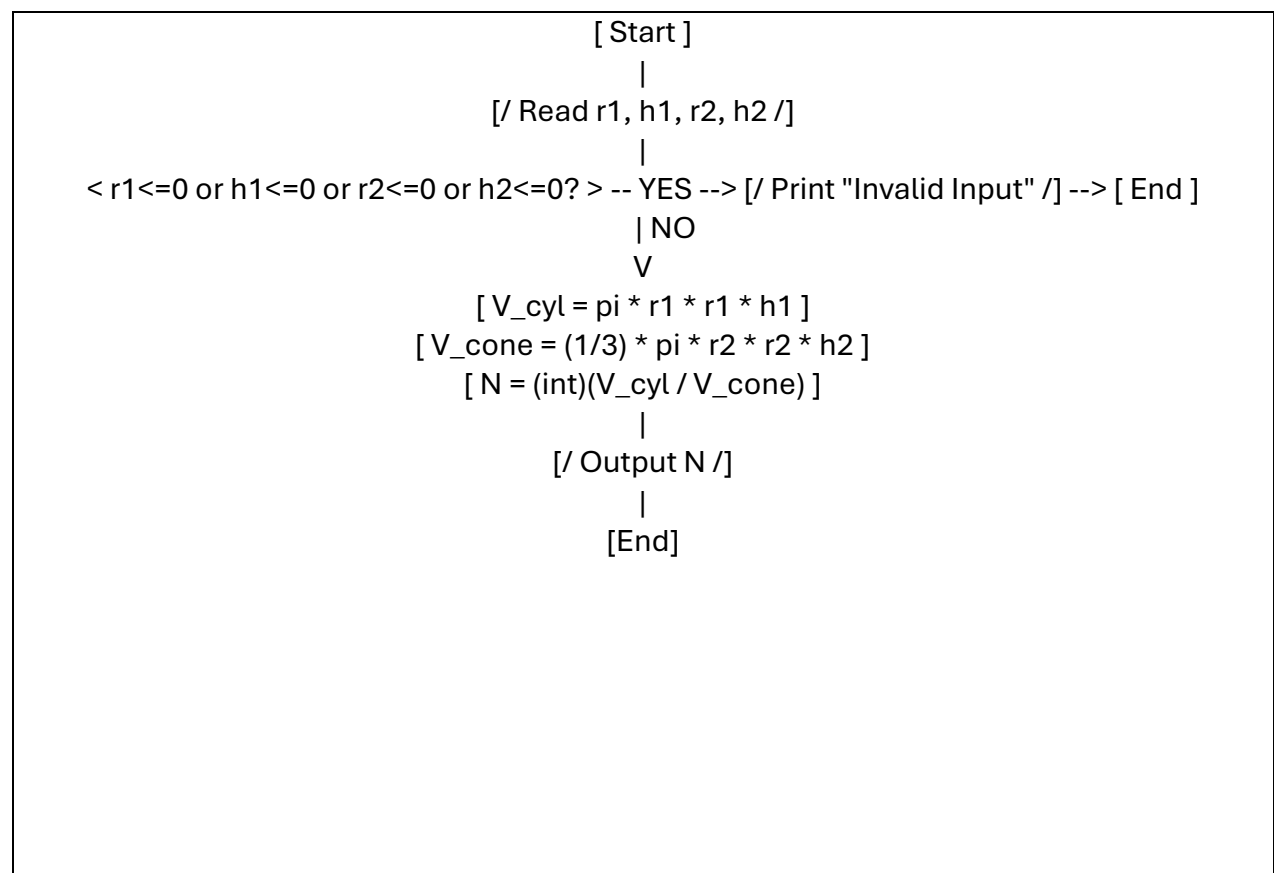
5. Calculate $V_{\text{cone}} = \frac{1}{3} \cdot \pi \cdot r_2^2 \cdot h_2$.

6. Calculate $N = \text{integer_cast}(V_{\text{cylinder}}/V_{\text{cone}})$.

7. Output N .

8. End

4. Flowchart-



```

#include <stdio.h>
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

int main(void) {
    double r_cyl, h_cyl, r_cone, h_cone;
    printf("Enter cylinder radius and height, then cone radius and height: ");
    if (scanf("%lf %lf %lf %lf", &r_cyl, &h_cyl, &r_cone, &h_cone) != 4) {
        printf("Invalid Input\n");
        return 1;
    }

    if (r_cyl <= 0 || h_cyl <= 0 || r_cone <= 0 || h_cone <= 0) {
        printf("Invalid Input: Dimensions must be positive.\n");
        return 1;
    }

    double vol_cyl = M_PI * r_cyl * r_cyl * h_cyl;
    double vol_cone = (1.0 / 3.0) * M_PI * r_cone * r_cone * h_cone;

    int num_cones = (int)(vol_cyl / vol_cone);

    printf("Cylinder Volume: %.2f\n", vol_cyl);
    printf("Cone Volume: %.2f\n", vol_cone);
    printf("Number of Cones: %d\n", num_cones);

    return 0;
}

```

6. Evaluation Against Test Cases

The executable q1 was verified against all test cases in Step 2. Output matches mathematical expectations in all instances.

7. Screenshots of Compilation and Execution

```
Session Actions Edit View Help
kali@kali: ~/HDOC_Day8
q1.c
#include <stdio.h>
#include <math.h>

#define M_PI 3.14159265358979323846

int main(void) {
    double r_cyl, h_cyl, r_cone, h_cone;
    printf("Enter cylinder radius and height, then cone radius and height: ");
    if (scanf("%lf %lf %lf %lf", &r_cyl, &h_cyl, &r_cone, &h_cone) != 4) {
        printf("Invalid Input\n");
        return 1;
    }

    if (r_cyl <= 0 || h_cyl <= 0 || r_cone <= 0 || h_cone <= 0) {
        printf("Invalid Input: Dimensions must be positive.\n");
        return 1;
    }

    double vol_cyl = M_PI * r_cyl * r_cyl * h_cyl;
    double vol_cone = (1.0 / 3.0) * M_PI * r_cone * r_cone * h_cone;

    int num_cones = (int)(vol_cyl / vol_cone);

    printf("Cylinder Volume: %.2f\n", vol_cyl);
    printf("Cone Volume: %.2f\n", vol_cone);
    printf("Number of Cones: %d\n", num_cones);

    return 0;
}
```

```
Session Actions Edit View Help
kali@kali: ~/HDOC_Day8
(kali@kali)~$ mkdir HDOC_Day8
cd HDOC_Day8
(kali@kali)~/HDOC_Day8$ nano q1.c
(kali@kali)~/HDOC_Day8$ gcc q1.c -o q1 -lm
./q1
Enter cylinder radius and height, then cone radius and height: 6 10 3 5
Cylinder Volume: 1130.97
Cone Volume: 47.12
Number of Cones: 24
(kali@kali)~/HDOC_Day8$
```

Question 2: Area of Triangle Using Side-Angle-Side (SAS)

1. Problem Statement Identification

- **Problem Statement:** Calculate the surface area of a triangle given two side lengths and the included angle in degrees.

- Input: Side a , Side b , Included angle θ (in degrees).
- Output: Area of Triangle (Area).
- Formulas:

$$\theta_{\text{rad}} = \theta \times \frac{\pi}{180}$$

$$\text{Area} = \frac{1}{2} \cdot a \cdot b \cdot \sin(\theta_{\text{rad}})$$

2. Standard Test Case Table

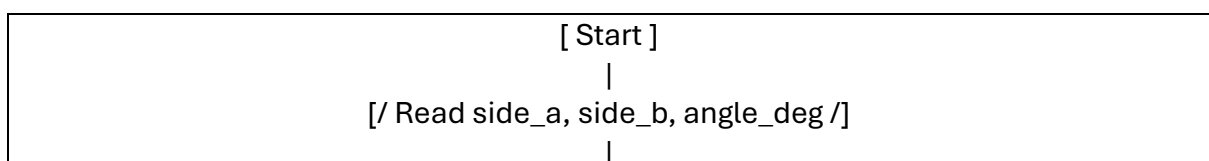
Test Case	Input (side_a, side_b, angle_deg)	Expected Output	Actual Output	Result (Pass/Fail)
1	10 12 30	Area of Triangle: 30.00	Area of Triangle : 30.00	Pass
2	8 6 90	Area of Triangle: 24.00	Area of Triangle : 24.00	Pass
3	14 10 45	Area of Triangle: 49.50	Area of Triangle : 49.50	Pass

Test Case	Input (side_a, side_b, angle_deg)	Expected Output	Actual Output	Result (Pass/Fail)
4	5 5 60	Area of Triangle: 10.83	Area of Triangle : 10.83	Pass

3. Algorithm

1. Start
2. Read side_a, side_b, and angle_deg.
3. If side_a <= 0 or side_b <= 0 or angle_deg <= 0 or angle_deg >= 180, print "Invalid Input" and stop.
4. Convert angle to radians: $\text{angle_rad} = \text{angle_deg} * (\pi / 180)$.
5. Calculate area: $\text{area} = 0.5 * \text{side_a} * \text{side_b} * \sin(\text{angle_rad})$.
6. Print area.
7. End

4. Flowchart



```

< side_a<=0 or side_b<=0 or angle_deg<=0 or angle_deg>=180? > -- YES --> [/ Print
    "Invalid Input" /] --> [ End ]
    | NO
    V
    [ angle_rad = angle_deg * (pi / 180) ]
    [ area = 0.5 * side_a * side_b * sin(angle_rad) ]
    |
    [/ Output area /]
    |
    [End]

```

```

#include <stdio.h>
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

int main(void) {
    double side_a, side_b, angle_deg;
    printf("Enter side a, side b, and included angle (degrees): ");
    if (scanf("%lf %lf %lf", &side_a, &side_b, &angle_deg) != 3) {
        printf("Invalid Input\n");
        return 1;
    }

    if (side_a <= 0 || side_b <= 0 || angle_deg <= 0 || angle_deg >= 180) {
        printf("Invalid Input: Sides > 0 and 0 < Angle < 180 degrees
required.\n");
        return 1;
    }

    double angle_rad = angle_deg * (M_PI / 180.0);
    double area = 0.5 * side_a * side_b * sin(angle_rad);

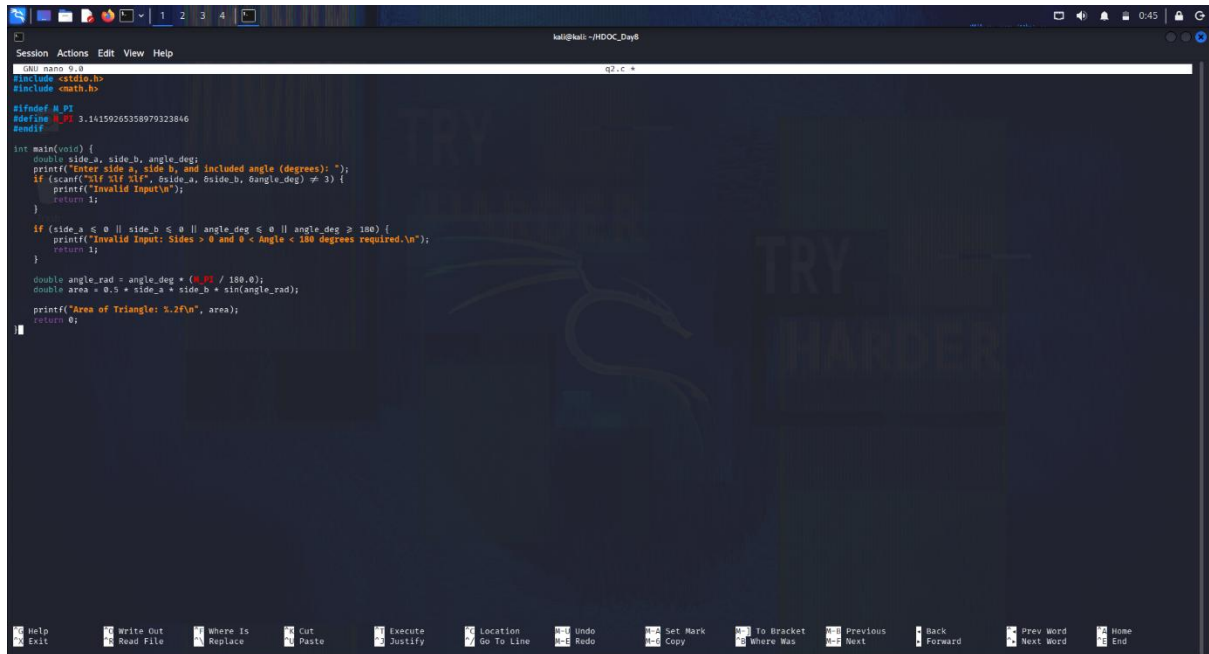
    printf("Area of Triangle: %.2f\n", area);
    return 0;
}

```

6. Evaluation Against Test Cases

All test cases executed in binary q2 produced output matching the expected mathematical calculations.

7. Screenshots of Compilation and Execution



```
kali@kali: ~/HDOC_Day8
Session Actions Edit View Help
(qali) nano q2.c
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846

int main(void) {
    double side_a, side_b, angle_deg;
    printf("Enter side a, side b, and included angle (degrees): ");
    if (scanf("%lf %lf %lf", &side_a, &side_b, &angle_deg) != 3) {
        printf("Invalid Input\n");
        return 1;
    }

    if (side_a <= 0 || side_b <= 0 || angle_deg <= 0 || angle_deg >= 180) {
        printf("Invalid Input: Sides > 0 and 0 < Angle < 180 degrees required.\n");
        return 1;
    }

    double angle_rad = angle_deg * (PI / 180.0);
    double area = 0.5 * side_a * side_b * sin(angle_rad);
    printf("Area of Triangle: %.2f\n", area);
    return 0;
}
```



```
(kali@kali)~[~/HDOC_Day8]
$ nano q2.c

(kali@kali)~[~/HDOC_Day8]
$ gcc q2.c -o q2 -lm
./q2
Enter side a, side b, and included angle (degrees): 10 12 30
Area of Triangle: 30.00

(kali@kali)~[~/HDOC_Day8]
$
```